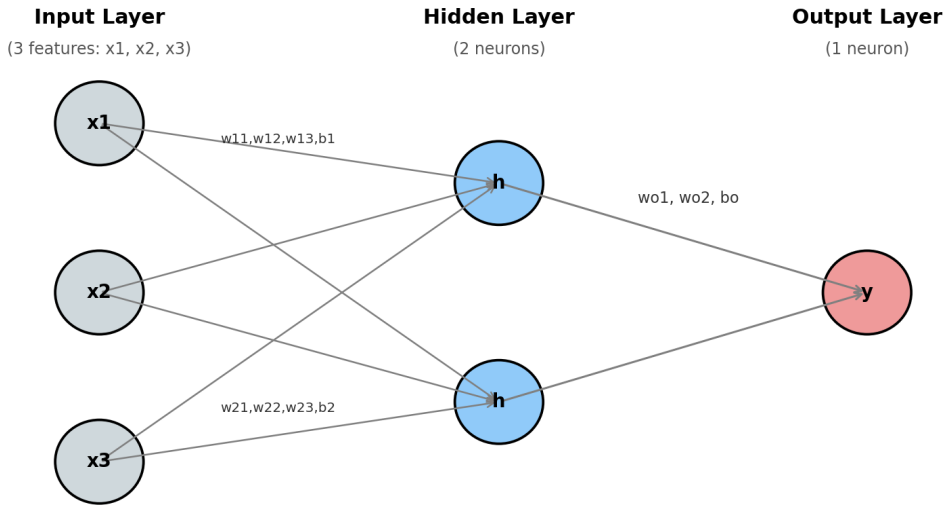
ANN Notes

Artificial Neural Networks — from architecture to full training pipeline

1. What is ANN? | 2. Basic Structure

- ANN = Artificial Neural Network — takes input numbers, passes them through layers of neurons, produces an output (prediction).
- Inspired loosely by brain neurons, but in code it's just math: multiplication + addition + activation function.
- Structure: **Input Layer -> Hidden Layer(s) -> Output Layer**



- **Input layer**: number of neurons = number of FEATURES per sample, NOT number of samples.
- **Hidden layer(s)**: where pattern detection happens. Number of layers/neurons is our choice (hyperparameter).
- **Output layer**: gives final prediction. Binary classification needs just 1 neuron with sigmoid.

Key rule: the SAME architecture handles ALL samples — one sample (or batch) at a time, reusing the same neurons/weights every time. Architecture grows with more FEATURES, never with more SAMPLES.

3. Weights & Biases | 4. Weight Notation

- Every neuron owns its OWN weight(s) and bias — never shared with other neurons, even in the same layer.
- **Why?** If neurons shared weights, they'd all learn the exact same pattern — extra neurons would be pointless.
- **Formula:** weights in a layer = (neurons in layer) x (inputs coming into layer)
- **Formula:** biases in a layer = (neurons in layer) — always 1 bias per neuron

Example:

Layer	Neurons	Inputs	Total Weights	Total Biases
Hidden	2	1 each	2	2
Output	1	2	2	1

Weights = one per INCOMING CONNECTION. Bias = one per NEURON. That's why the output layer above has 2 weights (2 incoming connections) but only 1 bias (only 1 neuron).

Weight notation: w(neuron)(input)

- **w11** = weight for neuron 1, connected to input 1
- **w21** = weight for neuron 2, connected to input 1
- First digit = which neuron. Second digit = which input it connects to.

5. Forward Pass | 6. Activation Functions

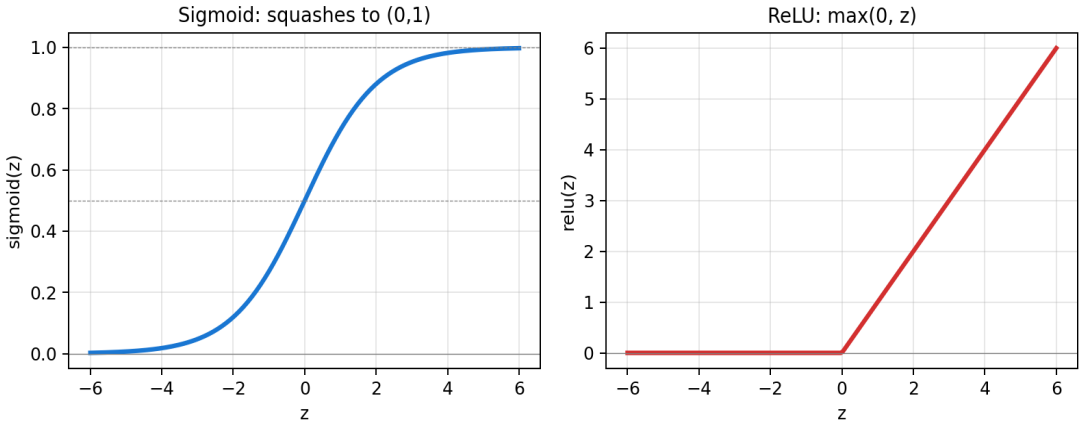
Step 1: Pre-activation

- z = w*x + b (raw weighted sum, BEFORE activation)

Step 2: Activation

- a = activation(z) (AFTER squashing through a function like sigmoid)

z and a are NOT the same. Always compute z first, then a. This repeats layer by layer — output of one layer becomes input to the next — until reaching the final prediction.



- **Sigmoid**: squashes any number to (0,1). Used in OUTPUT layer for binary classification (gives probability).
- **ReLU**: max(0, z) — negative becomes 0, positive stays same. Used in HIDDEN layers.
- **Why not sigmoid everywhere?** In deep hidden layers, sigmoid causes gradients to shrink layer by layer during backprop ("vanishing gradient"), slowing/stopping learning. ReLU avoids this.
- Without ANY activation function, stacking multiple layers mathematically collapses into just ONE linear layer — the network could only ever learn straight-line relationships.

7. Loss Function (BCE) | 8. Backpropagation

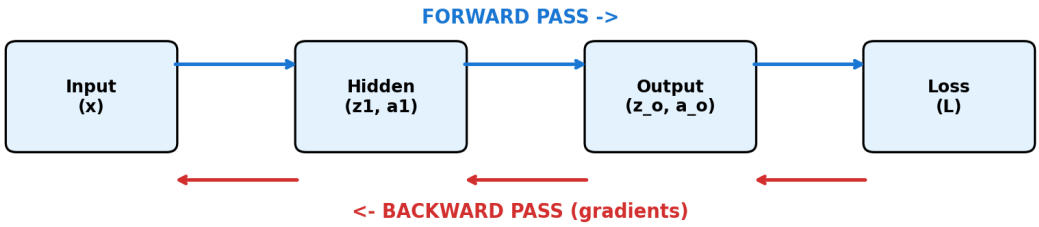
Binary Cross-Entropy formula:

```
L = -(1/n) * sum[ y*ln(y_hat) + (1-y)*ln(1-y_hat) ]
```

- y = true label (0 or 1), y_hat = predicted probability, n = number of samples
- **Why log, not simple subtraction?** BCE punishes CONFIDENT wrong answers much more heavily. If model predicts 0.99 but true answer is 0, loss shoots up sharply — a simple subtraction wouldn't capture this severity.

Backpropagation:

- Calculates how much EACH weight contributed to the final error, working BACKWARD from output to input, using the chain rule.
- Simplified shortcut (only for sigmoid + BCE together): **dL/dz_output = y_hat - y_true**
- This error then flows backward: dL/da(hidden) = dL/dz_output * weight, then multiplied by sigmoid's own derivative a*(1-a) to get dL/dz(hidden).



Important: backprop does NOT calculate new weights directly. It only calculates GRADIENTS (direction + magnitude). The actual weight update happens in a SEPARATE step — gradient descent.

9. Gradient Descent | 10. Epoch vs Batch

Weight update formula:

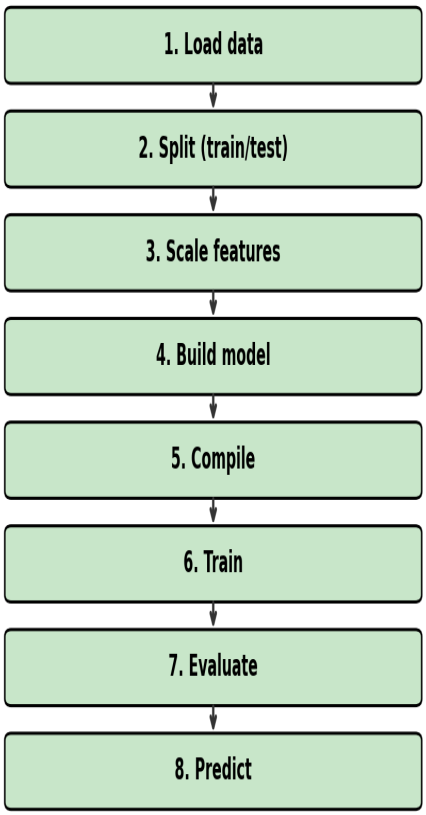
```
w_new = w_old - learning_rate * (dL/dw)

b_new = b_old - learning_rate * (dL/db)
```

- We SUBTRACT the gradient because gradient points toward INCREASING loss — we want the opposite direction, toward decreasing loss.
- Learning rate (alpha) controls step size: too high -> overshoots/bounces around. Too low -> trains painfully slow.

Epoch vs Batch:

- **Epoch** = one full pass through the ENTIRE training dataset.
- **Batch** = smaller chunk of data used for ONE forward + backward + update step.
- Example: 1000 samples, batch_size=100 -> 10 batches per epoch -> 10 weight updates in ONE epoch.
- Weight updates happen once per BATCH, not once per epoch (unless batch size = full dataset — called "full-batch" gradient descent).

11. Full Real-World Training Pipeline



- **Load data**: check shape, feature names, class balance, missing values.
- **Split (train/test)**: random_state for reproducibility, stratify for balanced classes.
- **Scale features**: fit_transform on train, transform ONLY on test.
- **Build model**: Input layer -> Hidden layers -> Output layer.
- **Compile**: choose optimizer (e.g. Adam) + loss function.
- **Train**: model.fit(), watch loss / val_loss / accuracy.
- **Evaluate**: model.evaluate() on test set — forward pass only, no training.
- **Predict**: new sample must be reshaped + scaled with the SAME scaler before predicting.

random_state vs stratify:

- **random_state**: seed number for the shuffle. Same seed = same shuffle order every independent run — deterministic math, not "memory."
- **stratify**: ensures train AND test sets keep the SAME class balance/ratio as the full dataset. Without it, an imbalanced dataset risks a skewed split.

fit_transform vs transform:

- **fit_transform**: LEARNS the scaling pattern (mean/std) AND applies it — TRAINING data only.

- **transform** (alone): only APPLIES an already-learned pattern — TEST data / new data only. Prevents data leakage.

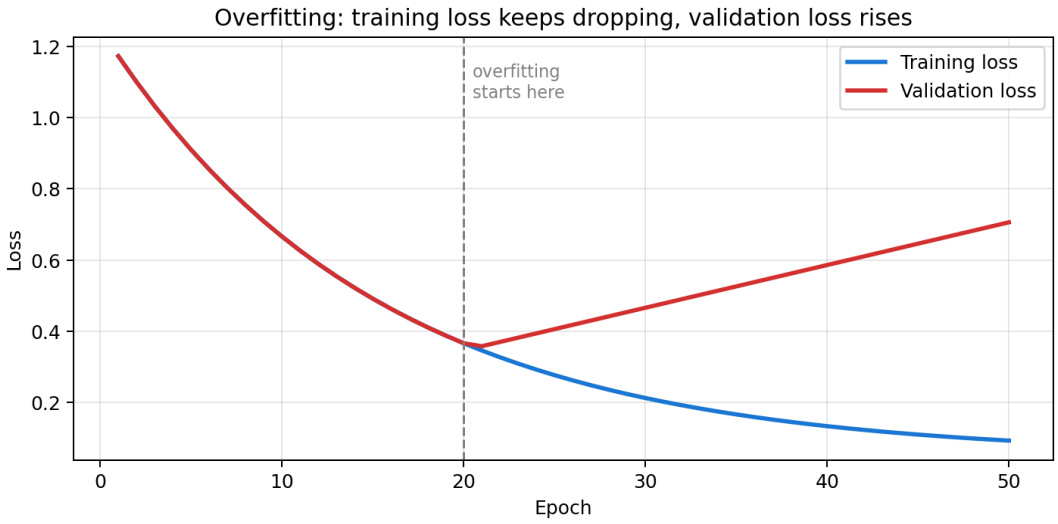
12. Optimizers | 13. Overfitting

SGD vs Adam:

- **SGD**: same fixed learning rate for every weight, every step. Fine for tiny problems.
- **Adam**: adapts learning rate individually per weight based on recent gradient history (momentum + adaptive scaling). Faster, more stable — standard choice for real datasets.

Overfitting:

- Model memorizes training data instead of learning patterns that generalize to new data.
- **How to spot it**: training accuracy very high, test/validation accuracy much lower. Or training loss keeps dropping while validation loss starts rising.
- **Fixes**: reduce model complexity, get more data, early stopping, regularization/dropout.



14. Quick Reminders — Rules to Never Forget

- Input neurons = number of FEATURES, not number of SAMPLES.
- z = before activation, a = after activation. Always write both separately.
- A weight belongs to the CONNECTION (or neuron), NOT to individual samples — same weight reused across all samples in one forward pass, changes only between epochs.
- Backprop only computes GRADIENTS. Gradient descent is the separate step that updates weights.
- fit_transform = train data only. transform = test/new data only. Never mix these up.
- random_state = reproducible shuffle. stratify = balanced class split. Two different jobs.